

Unidad 4: Git avanzado

Esta semana nos sumergiremos en la "física" de Git para transformar la herramienta de un simple guardador de archivos en una infraestructura de ingeniería resiliente. Comenzaremos desarmando el motor interno para comprender cómo la base de datos de objetos (**blobs**, **trees** y **commits**) garantiza la integridad de los datos, lo que nos permitirá diagnosticar y reparar problemas complejos que bloquean el desarrollo. Posteriormente, aprenderemos a "curar" el historial mediante técnicas de limpieza como **squash** y **amend**, asegurando una línea de tiempo profesional y legible, apoyándonos en herramientas de recuperación como **reflog** para revertir desastres accidentales. Finalmente, dominaremos la manipulación atómica de cambios con comandos de precisión como **cherry-pick** y **stash**, permitiéndonos mover funcionalidades de forma quirúrgica entre ramas sin comprometer la estabilidad del proyecto ni perder el progreso actual.

1. ¿Que es GIT?

Git es un **sistema de control de versiones distribuido** usado en desarrollo de software para **registrar cambios en archivos a lo largo del tiempo**. Permite que varias personas trabajen sobre el mismo proyecto sin perder el historial de modificaciones.

Fue creado en 2005 por **Linus Torvalds**, el mismo creador del sistema operativo **Linux**.

Git permite principalmente:

1. **Guardar el historial de cambios**

   Cada modificación en el código queda registrada.

2. **Trabajar en equipo**

   Varias personas pueden editar el mismo proyecto sin pisarse el trabajo.

3. **Volver a versiones anteriores**

   Si algo se rompe, se puede recuperar una versión anterior del proyecto.

4. **Trabajar en paralelo**

   Con “ramas” (*branches*), distintos desarrolladores pueden probar cosas sin afectar el código principal.

2. Sistemas Centralizados vs. Descentralizados

Antes de analizar las herramientas concretas de control de versiones, conviene distinguir entre dos modelos de arquitectura muy utilizados en los sistemas informáticos: **centralizado** y **descentralizado**. En un sistema centralizado, existe un único servidor o nodo principal que concentra la información y coordina las operaciones, mientras que los usuarios o clientes dependen de él para acceder a los datos. En cambio, en un sistema descentralizado la información y las funciones se distribuyen entre múltiples nodos, de modo que cada uno puede operar de forma relativamente autónoma. Esta diferencia arquitectónica también se refleja en los sistemas de control de versiones.

- **Sistemas de Control de Versiones Centralizados (CVCS):** En herramientas como Subversion (SVN) o Perforce, existe un único servidor central que contiene todas las versiones de los archivos. Los clientes descargan únicamente una versión específica (la "copia de trabajo") del servidor.
    - **Riesgo:** Si el servidor central falla o el repositorio se corrompe, se pierde el historial completo si no hay copias de seguridad externas. La dependencia de la red es constante para realizar cualquier operación de historial.
- **Sistemas de Control de Versiones Descentralizados (DVCS):** En Git, cada colaborador posee una **copia local completa** del repositorio, incluyendo el historial íntegro de commits y ramas.
    - **Ventaja:** La mayoría de las operaciones son locales y no requieren conexión a internet. El servidor actúa meramente como un punto de

sincronización, pero cada terminal de desarrollador funciona como un respaldo total del proyecto.

3. El Modelo de Deltas vs. Instantáneas (Snapshots)

Gestión por Deltas (Basada en Cambios)

Muchos sistemas tradicionales (como CVS o SVN) almacenan la información como una lista de cambios. Este método guarda el archivo original y luego una serie de "parches" o diferencias (*deltas*) que representan las modificaciones línea por línea.

- **Proceso:** Para reconstruir la versión actual de un archivo, el sistema debe computar y aplicar secuencialmente todos los deltas acumulados desde la base original.

Gestión por Instantáneas (Modelo de Git)

Git no almacena el historial como una suma de diferencias. En su lugar, cada vez que se realiza una confirmación (*commit*), Git captura el **estado total** del sistema de archivos en ese instante.

- **Eficiencia de Almacenamiento:** Si un archivo no ha sido modificado entre dos versiones, Git no guarda el archivo nuevamente; simplemente crea un enlace o referencia al objeto de datos (blob) idéntico que ya existe en su base de datos.
- **Integridad:** Este modelo permite que Git conozca el contenido exacto de cada versión de forma inmediata, sin necesidad de recalcular diferencias, facilitando operaciones como el cambio de ramas (*branching*) y la fusión de código (*merging*).

4. La Infraestructura de Datos: Objetos y Direccionamiento

Para materializar el modelo de instantáneas de forma eficiente, Git implementa un sistema de **almacenamiento direccionable por contenido**. Bajo este esquema, el identificador de un archivo deja de ser su nombre o ruta para

convertirse en una "huella digital" (Hash SHA-256[1]) calculada estrictamente a partir de sus datos. Esto significa que la "dirección" de un archivo en la base de datos es su contenido mismo: si el contenido no varía entre versiones, el hash resultante es idéntico, permitiendo que Git reutilice el mismo objeto físico en el disco para múltiples momentos del proyecto. Esta arquitectura no solo optimiza radicalmente el espacio mediante la deduplicación, sino que garantiza que los datos sean inmutables, ya que cualquier intento de alteración cambiaría su hash y, por lo tanto, su dirección lógica.

4.1 El Modelo de Objetos

La base de datos interna (ubicada en `.git/objects`) organiza esta información en una estructura jerárquica de cuatro objetos atómicos:

- **Blobs (Binary Large Objects):** Representan el nivel más bajo, almacenando el contenido binario de los archivos. Al ser direccionables por contenido, carecen de metadatos como el nombre o los permisos; son simplemente datos puros identificados por su hash.
- **Trees (Árboles):** Actúan como la capa de organización y contexto. Un *Tree* asocia los nombres de archivos y modos de permisos con sus respectivos hashes de *Blobs* (o con otros *Trees* en caso de subdirectorios), reconstruyendo así la jerarquía de carpetas a partir de objetos planos.
- **Commits (Unidades de Historial):** Consolidan un punto específico en la línea de tiempo. Cada objeto *Commit* vincula un *Tree* raíz (el estado total del proyecto) con metadatos de autoría y, fundamentalmente, con el hash del commit o los commits antecesores.
- **Tags (Referencias Estáticas):** Son punteros inmutables que apuntan a un commit específico para marcar hitos o lanzamientos de versión.

4.2 El Grafo Dirigido Acíclico (DAG)

[1]Aunque Git nació con SHA-1, las versiones actuales permiten el uso de SHA-256 para mayor seguridad contra colisiones de hash



La relación de parentesco entre los commits no forma una lista lineal, sino un **Grafo Dirigido Acíclico (DAG)**. En esta estructura, cada commit es un nodo que apunta hacia atrás, hacia su padre o padres (en el caso de los merges).

Esta arquitectura es el pilar de la **Integridad Total** de Git. Debido a que el hash de cada commit se calcula incluyendo el identificador de su predecesor, se crea una cadena de custodia criptográfica: cualquier alteración mínima en un *Blob* del pasado cambiaría su hash, invalidando el *Tree* que lo contiene y, por efecto cascada, alterando los hashes de todos los *Commits* sucesores. Este diseño asegura que el historial sea auditable y virtualmente imposible de falsificar sin romper la integridad de la cadena completa.

5. Las Tres Áreas de Git y el Flujo de Trabajo

Para gestionar las instantáneas de forma segura, Git no guarda los cambios directamente en el repositorio. En su lugar, utiliza un sistema de "preparación" que permite al desarrollador construir el siguiente estado del proyecto de forma atómica.

5.1 Definición de las Áreas

1. **Working Directory (Directorio de Trabajo):** Es la carpeta física en tu sistema de archivos donde están tus archivos reales. Aquí es donde los modificas, borras o creas. En esta área, los archivos son simples datos en el disco fuera del control estricto de la base de datos de Git.
2. **Staging Area o Index (Área de Preparación):** Es un archivo técnico (ubicado en `.git/index`) que actúa como un borrador del próximo commit. Cuando "agregas" un archivo, Git prepara su entrada en este índice. Es la zona donde decides qué cambios están "listos para la foto".
3. **Repository (Repositorio / .git):** Es la base de datos permanente donde Git almacena los objetos (blobs, trees, commits) y las referencias (ramas, tags). Una vez que los datos llegan aquí, forman parte del historial inmutable.

5.2 El Ciclo de Vida de una Modificación

El proceso técnico al ejecutar comandos básicos es el siguiente:

- **`git add <archivo>`:** Al ejecutar este comando, Git lee el contenido del archivo en el **Working Directory**, calcula su Hash SHA-256 y lo guarda inmediatamente como un **Blob** en la base de datos de objetos. Luego, actualiza el **Index** para que el nombre del archivo apunte a ese nuevo hash.
- **`git commit`:** Git toma el estado actual del **Index**, crea un objeto **Tree** con esa estructura y luego genera un objeto **Commit** que apunta a ese árbol. En este momento, la referencia de la rama actual (ej. `main`) se actualiza para apuntar al hash de este nuevo commit.
- **`git checkout`:** Realiza el proceso inverso. Git busca el commit solicitado en el **Repository**, extrae su estructura de árboles y blobs, y los "vuelca" físicamente en tu **Working Directory**, actualizando también el **Index** para que coincida con ese estado.

5.3 El rol de HEAD

En este flujo, **HEAD** es el puntero que indica en qué parte del grafo de commits estás trabajando actualmente.

- Casi siempre, HEAD apunta a una **rama** (que a su vez apunta al commit más reciente).
- Cuando haces un commit, la rama avanza y HEAD se mueve con ella.
- Entender HEAD es vital para la **resiliencia**, ya que nos permite "despegar" nuestra vista de la rama actual para explorar estados pasados sin alterar el historial.

6. Gestión del Historial con Precisión

En un entorno profesional (especialmente cuando trabajamos con **IA**, que puede generar múltiples cambios rápidos), el historial puede volverse ruidoso. La gestión de precisión nos permite "limpiar" el pasado antes de integrarlo a la rama principal.

6.1 Técnicas de Limpieza y Corrección

- **`git commit --amend` (Corrección Inmediata):** Se utiliza para modificar el último commit realizado. Técnicamente, Git no edita el commit anterior, sino que crea uno nuevo con los cambios corregidos y **reemplaza** al anterior en la punta de la rama. Es ideal para corregir errores de dedo en el mensaje o incluir un archivo que olvidamos añadir al *staging area*.
- **`git rebase -i` (Rebase Interactivo / Squash):** Es la herramienta de "cirugía estética" del historial. Permite tomar una serie de commits y transformarlos.
    - **Squash:** Combina varios commits en uno solo. Útil para unir micro-commits (ej: "arreglo 1", "arreglo 2", "test") en una sola unidad funcional coherente.
    - **Reword:** Cambia el mensaje de commits antiguos para que sigan estándares como *Conventional Commits*.

6.2 Resiliencia Técnica: Recuperación de Estados

La resiliencia en Git no es magia; es una consecuencia de su arquitectura. Al ser un sistema que añade datos constantemente (*append-only*), Git casi nunca borra objetos de su base de datos de inmediato. Esta sección cubre cómo recuperar lo que parece perdido y cómo deshacer cambios de forma segura según el contexto.

A. El Mapa de Rescate: git reflog

Mientras que `git log` muestra la historia "oficial" de una rama, el **reflog** es el historial de movimientos del puntero **HEAD**.

- **Su valor:** Si realizas un `reset --hard` accidental y tus commits "desaparecen" de la vista, el reflog conserva los hashes de todos los lugares donde estuvo parado tu HEAD.
- **Persistencia:** Los commits huérfanos (que no pertenecen a ninguna rama) permanecen en la base de datos hasta que Git ejecuta su **Garbage**



Collector (gc), lo que suele ocurrir tras 30 o 90 días. Tienes una ventana de tiempo amplia para rescatarlos.

B. Moviendo el Puntero: git reset

El comando `reset` se utiliza para retroceder en el tiempo moviendo el puntero de la rama actual a un commit específico. Se usa principalmente en el **historial local**.

- **`--soft`:** Mueve el puntero pero mantiene tus cambios en el *Staging Area*. Es ideal para deshacer un commit y volver a prepararlo.
- **`--hard`:** Mueve el puntero y sobreescribe físicamente tu directorio de trabajo. **Peligro:** Borra los cambios actuales del disco, por lo que solo es seguro si sabes que puedes volver atrás mediante el reflog.

C. La Alternativa Segura: git revert

A diferencia de `reset`, que "mueve" el pasado, **`revert` crea un nuevo futuro**.

- **Mecánica:** Genera un nuevo commit que aplica exactamente los cambios inversos al commit que quieres anular.
- **Resiliencia en equipo:** Es la forma **única y segura** de deshacer cambios en ramas compartidas o remotas (`main`, `develop`). Al no alterar el historial existente, no rompe el trabajo de tus compañeros. Mientras que `reset` borra, `revert` documenta el error y su corrección.

Situación	**Comando recomendado**	**Impacto en el Historial**
Error en el último commit local	git commit --amend	Sobreescribe el último commit.
Borré commits	git reflog + git reset	Recupera commits

Situación	Comando recomendado	Impacto en el Historial
accidentalmente		"perdidos".
Quiero limpiar mi rama local	git reset --hard	Mueve el puntero y limpia archivos.
Error en un commit ya subido (push)	git revert	Añade un commit de reversión (Seguro).

6.3 Manipulación Atómica: Cherry-pick y Stash

La atomicidad busca que cada commit sea una unidad mínima e independiente.

- **`git cherry-pick <hash>`:** Permite copiar un commit específico de una rama y aplicarlo en la rama actual. Es fundamental cuando necesitas una corrección urgente (*bugfix*) que está en una rama de desarrollo inestable, sin tener que hacer un `merge` de toda la rama.
- **`git stash` (La zona de pausa):** Guarda temporalmente los cambios del directorio de trabajo y el índice en una pila interna, dejando el repositorio limpio.
    - **Uso con IA:** Si la IA te sugiere un cambio radical, puedes hacer un `stash` de tu trabajo actual, probar la sugerencia en una rama limpia, y luego hacer un `stash pop` para recuperar tus cambios originales si la prueba no funciona.

7. Caso Práctico de Resiliencia: El Rescate del Commit Perdido

Imagina que estás trabajando en una funcionalidad compleja. Haces un commit (`Hash: A1B2`), pero luego decides probar algo distinto. Ejecutas un `git reset --hard HEAD~1` pensando que era lo correcto, pero te das cuenta de que has borrado tus cambios del disco y del historial.



Paso 1: El pánico inicial

Si haces `git log`, el commit `A1B2` ya no aparece. La rama ha retrocedido y tus archivos han vuelto al estado anterior.

Paso 2: El uso de reflog (La Caja Negra)

Git no borra los objetos inmediatamente. Aunque la rama no apunte a `A1B2`, el registro de movimientos del puntero `HEAD` sí lo tiene. Ejecutas: `git reflog`

Verás una lista similar a esta:

- `c3d4e5f HEAD@{0}: reset: moving to HEAD~1`
- `a1b2c3d HEAD@{1}: commit: Implementación de lógica compleja` <-- **¡Aquí está!**

Paso 3: La Restauración

Ahora que tienes el hash (`a1b2c3d`), puedes "resucitar" ese estado. Tienes dos opciones:

1. **Crear una rama nueva ahí:** `git branch rescate a1b2c3d`
2. **Forzar la rama actual a volver:** `git reset --hard a1b2c3d`

8. Buenas Prácticas de Manipulación Atómica

Para evitar tener que usar el protocolo de rescate con frecuencia, la manipulación atómica es tu mejor aliada:

- **Commits Pequeños:** Si usas IA, esta suele generar muchos cambios. Usa `git add -p` (patch mode) para revisar cada bloque de código sugerido. Esto te permite aceptar solo las partes que entiendes y funcionan, creando commits con una única responsabilidad.
- **Stash con Mensajes:** No hagas solo `git stash`. Usa `git stash push -m "Intento de refactor con IA"` para que, si tienes varios elementos guardados, sepas exactamente cuál recuperar después.



- **Cherry-pick para Hotfixes:** Si descubres un error crítico en producción mientras trabajas en una rama nueva, no mezcles todo. Haz el arreglo en una rama aparte, haz commit, y luego usa `cherry-pick` para llevar ese arreglo específico a `main` y a tu rama actual.

9. Diagnóstico y Resolución de Problemas Complejos

El conocimiento del modelo de objetos no es solo teórico; es la herramienta principal para auditar un repositorio cuando las herramientas automáticas fallan.

9.1 Inspección de la Base de Datos (Diagnóstico)

Cuando un repositorio se comporta de forma inesperada o sospechas de corrupción de datos, puedes "abrir el capó" con comandos de bajo nivel (*plumbing commands*):

- **`git cat-file -t <hash>`:** Te dice el **tipo** de objeto (blob, tree o commit) que se esconde tras un hash. Es el primer paso para entender una referencia rota.
- **`git cat-file -p <hash>`:** Muestra el **contenido** del objeto. Si es un *tree*, verás la lista de archivos; si es un *blob*, verás el código puro. Esto permite recuperar contenido incluso si el archivo fue borrado y no recuerdas el nombre.
- **`git ls-tree HEAD`:** Muestra la estructura jerárquica que HEAD está viendo en este momento, permitiendo diagnosticar por qué ciertos archivos no se están incluyendo en el snapshot.

9.2 El Concepto de Atomicidad en la Práctica

La manipulación atómica garantiza que cada commit sea una unidad funcional mínima. Esto es vital para que herramientas como `cherry-pick` funcionen sin generar conflictos masivos.



- **Integridad del Código con `stash`:** El `stash` no es solo un "portapapeles". Al usar `git stash pop`, Git intenta realizar un **merge** de tus cambios temporales sobre el estado actual. Si hay conflictos, Git te obliga a resolverlos antes de seguir, protegiendo la integridad de tu directorio de trabajo.
- **Cherry-pick como herramienta de auditoría:** Al traer un commit suelto de otra rama, Git verifica si el "Tree" de origen es compatible con tu "Tree" actual. Si la arquitectura interna detecta que el cambio depende de código que no tienes, el proceso se detiene, evitando que introduzcas bugs por dependencias faltantes.

9.3 Manteniendo una Línea de Tiempo Profesional

El resultado de aprendizaje de "mantener una línea de tiempo profesional" se logra mediante la combinación de limpieza y recuperación:

1. **Limpieza Preventiva:** Antes de compartir código, se usa `rebase -i` para que la historia sea una progresión lógica (Ej: de `A -> B -> C` en lugar de `A -> corrección -> test -> B`).
2. **Recuperación Post-Error:** Si en la limpieza borras un commit importante, el `reflog` te permite saltar al hash exacto antes de la limpieza.

Regla de Oro: Se puede "reescribir el pasado" (amend, squash, rebase) siempre y cuando los commits sean **locales**. Una vez que el código se ha enviado al repositorio remoto (`push`), el historial se vuelve sagrado para no romper el trabajo de los demás.